Programming Assignment 2: Structure from Motion

202511013 권세현

https://github.com/tpgusrnjs/CSE404/tree/main/pa2

1 Introduction

Structure from Motion은 서로 다른 시점에서 촬영된 여러 장의 2D 이미지로부터 3D 구조와 카메라의 위치 및 자세를 동시에 복원하는 기법이다. 본 과제에서는 세 장의 이미지로부터 sparse 3D point cloud와 각 카메라의 pose를 추정하는 것을 목표로 한다.

SfM의 핵심은 서로 다른 이미지 간의 feature correspondence를 이용하여 기하학적 관계를 복원하는 것이다. 두 장의 이미지로 초기 reconstruction을 수행하고, 세 번째 이미지를 통해 PnP 기반으로 구조를 확장한다

2 Method

2.1 T1: SIFT matching

두 이미지 간의 대응점을 찾기 위해 cv2.SIFT create()를 사용하였다. SIFT는 scale과 rotation에 invariant한 특징점을 검출하고, 각 keypoint에 대해 128차원의 descriptor를 생성한다. 두 이미지의 descriptor d_i, d_j'에 대해 Euclidean distance를 기준으로 nearest neighbor를 찾고, Lowe's ratio test를 적용하였다.

2.2 T2: Essential matrix

카메라의 상대적인 위치와 방향을 추정하기 위해 Essential matrix E가 필요하다. 정규화된 이미지 좌표 $\tilde{x}, \tilde{x}'$는 다음과 같이 계산된다: $\tilde{x} = K^{-1}x$ 이때 epipolar constraint는 다음과 같다.

$$\tilde{x}'^T E \tilde{x} = 0$$

본 구현에서는 cv2.findEssentialMat()을 사용하여 RANSAC 기반으로 E를 추정하였다. Threshold는 0.005로 설정하였다.

	Image0 and Image1	Image0 and Image2
RANSAC inlier ratio	445/478 = 93.10%	389/445 = 87.42%

2.3 T3: Pose disambiguation

Essential matrix E는 두 카메라 간의 상대적 회전 R과 평행이동 t를 포함한다. $E = [t] \times R$ 여기서 $[t]_\times$는 벡터 t의 skew-symmetric matrix이다. E를 SVD 하면 $E = U\Sigma V^T$이며 $\Sigma = diag(s, s, 0)$를 만족해야 한다. (단 $s > 0$) Σ를 skew-symmetric matrix의 표준형인 Z로 변환하기 위해 Z축 90° 회전 행렬 W를 도입하면, E의 Null space를 형성하는 U의 u_3는 $[t]_\times u_3 = 0$을 만족하므로, 평행 이동 벡터는 방향성에 따라 $t = \pm u_3$의 두 가지 후보로 결정된다. 즉, t와 $-t$는 동일한 epipolar constraint를 만족하므로 구별할 수 없다. 또한 $E = (UZU^T)(UW^TV^T)$에서 가능한 회전경로가 $R = UWV^T$ 또는 $R = UW^TV^T$로 두 가지가 주어진다.

따라서 Essential matrix SVD를 수행하면 총 4개의 가능한 (R, t) 조합이 생성된다. 그러나 실제 물리적으로 가능한 해는 하나뿐이며, 선택하기 위해 cheirality condition을 사용했다. triangulation된 3D 점을 두 카메라 모두에서 depth 계산하여, 양의 깊이의 개수를 확인하였다. 가장 많은 점이 양의 깊이를 가지는 (R, t)를 최종 pose로 선택하였다.

2.4 T4: Triangulation

두 카메라에서 관측된 대응점으로부터 3D 좌표를 복원하기 위해 Direct Linear Transformation를 사용한다. 각 대응점 (x, x')에 대해 다음 선형 시스템을 구성한다. $AX = 0$

$$A = \begin{pmatrix} xp_3^T - p_1^T \\ yp_3^T - p_2^T \\ x'p_3'^T - p_1'^T \\ y'p_3'^T - p_2'^T \end{pmatrix}$$

homogeneous 좌표를 정규화하여 3D 점 X를 얻는다. 이후 reprojection error는 $\epsilon = \|x - \pi(PX)\|^2 + \|x' - \pi(P'X)\|^2$로 계산되며, threshold를 2px로 설정하여 outlier 3D점을 제거하였다.

	Image0 and Image1	Image0 and Image2
points kept	444/445	385/389
Mean reprojection error	0.205 px	0.289 px

2.5 T5: PnP registration

세 번째 이미지에 대해 기존 3D 점과의 correspondence를 이용하여 카메라 pose를 추정하였다.

$$\min_{R,t} \sum_{j=1}^{N} |x_j - \pi\big(K[\,R \mid t\,]X_j\big)|^2$$

본 구현에서는 EPnP와 cv2.solvePnPRansac()을 사용하였다. 첫 번째 이미지에서 생성된 3D 점과 SIFT descriptor index를 기반으로 correspondence를 유지한 것이다. 이후 새로운 view와 기존 view를 이용하여 추가적인 3D 점을 triangulation하여 point를 확장하였다.

3 Result

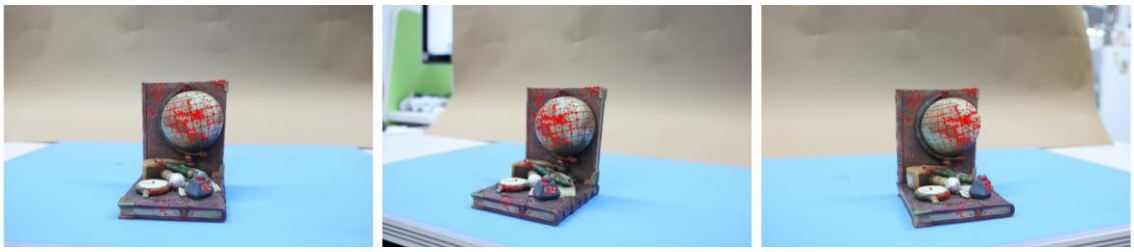

Figure 1 each image represents reprojection points

PnP matches는 134개이며 Total 3D points는 829개이다.

4 Discussion

이미지 위에 다시 projection 해보면 feature점과 reprojection점이 미묘하게 어긋난 것을 관찰할 수 있었다. 이는 outlier 존재하며 더 정확한 규제 방법이 필요하다는 것을 의미한다.

본 과제를 구현하면서 체감한 점은 작은 차이가 결과에 큰 영향을 미친다는 것이었다. 특히 dehomogenization를 하지 않고 진행했을 때 reprojection error에서 이상치가 증가하는 현상을 확인할 수 있었다. 또한 파이프라인을 가능한 수식 기반으로 직접 구현하려고 시도했지만, 실제로는 코드 수준에서 수치적으로 안정하게 구현하는 것이 생각보다 훨씬 어렵다는 점을 느꼈다.

5 Conclusion

본 과제에서는 SIFT feature matching, Essential matrix, pose 복원, triangulation, PnP를 통한 multi-view 확장을 수행하였다. 이를 통해 2D 이미지로부터 3D 구조를 성공적으로 복원할 수 있음을 확인하였다. 향후에는 bundle adjustment를 적용하여 전체 구조를 global하게 최적화하면 정확한 reconstruction을 얻을 수 있을 것으로 기대된다.